universität wien

MULTIOMICS DATA SCIENCE - (TIME) SERIES ANALYSIS

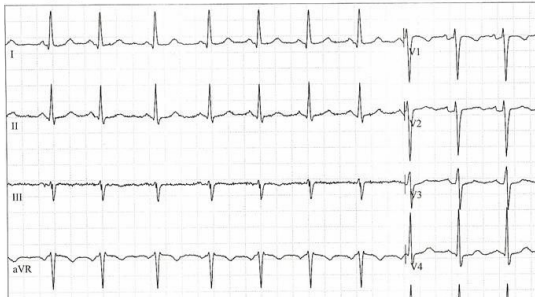
(TIME)SERIES

- A series is a (sorted) collection of dependent numerical values
- A time series maps time points (x axis) on another (physical) dependent value with arbitrary unit

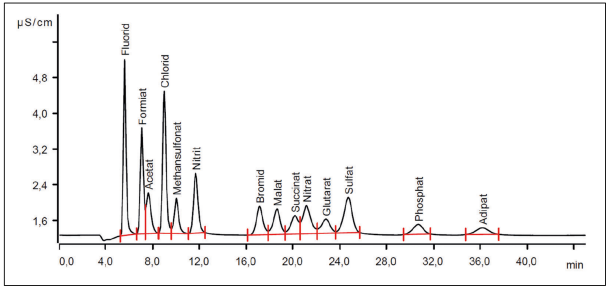
(TIME)SERIES

- A series is a (sorted) collection of dependent numerical values
- A time series maps time points (x axis) on another (physical) dependent value with arbitrary unit
- Examples:
  - Stock price
  - Electrocardiogram (EKG)
  - Chromatograms

Quelle: https://www.finanzen.at/chart/tesla



Quelle: https://de.wikipedia.org/wiki/Elektrokardiogramm



Quelle: https://www.all-electronics.de

universität wien

OTHER SERIES

- Numerical series, examples:
  - Weight
  - Length
  - Frequences

- Set with independent values:
  - Body heights of 1000 people
  - Concentrations of 4000 identified proteins

- Nominal / categorial values:
  - Weight: below average, average, above average, obese
  - Times: morning, noon, afternoon, evening
  - Sizes: small, medium, large, extra large

ANALYSIS APPROACHES ON GENERIC SERIES

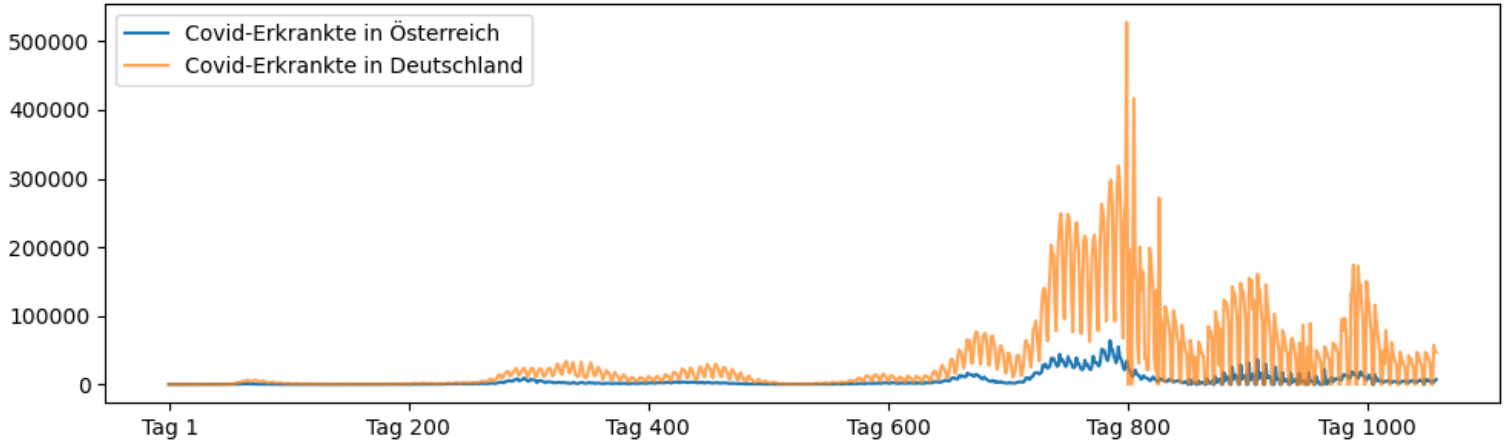
- Dependent on the type of data, only certain analysis methods can be applied
- Methods with arithmetic operations can only be applied on numerical data
- When using comparing methods, nominal data with defined order (e.g. small < medium < large) can be used

NOISY DATA

- Data are often biased with noise (fluctuation by empirial surveys, or from physical detectors)
- Noisy data can only be compared with each other to a limited extent
- One method for improving comparability is to smooth the data

NOISY DATA

- Data are often biased with noise (fluctuation by empirial surveys, or from physical detectors)
- Noisy data can only be compared with each other to a limited extent
- One method for improving comparability is to smooth the data

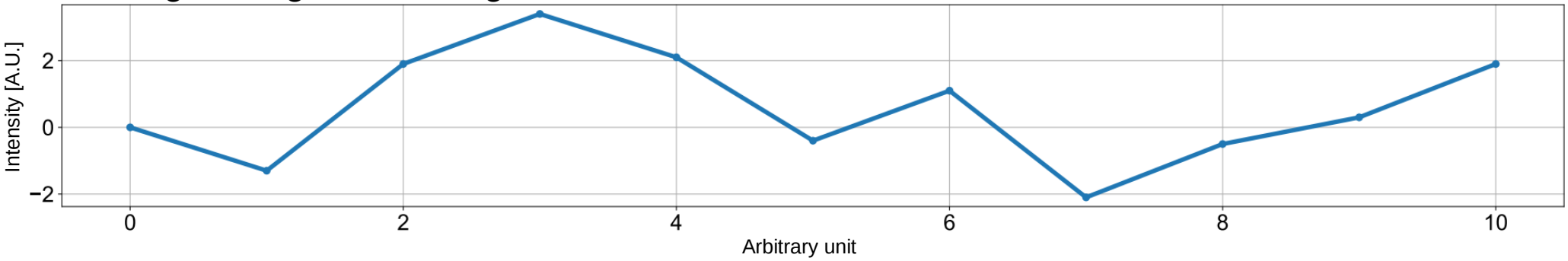


DATA SMOOTHING

- Data smoothing is often a local adjustment of data points dependent on their local neighboring data
- The easiest method for smoothing is the „moving average smoothing“
- When considering a data point, the average of all data points withing a defined window will be assinged to the current data point

DATA SMOOTHING

moving average smoothing

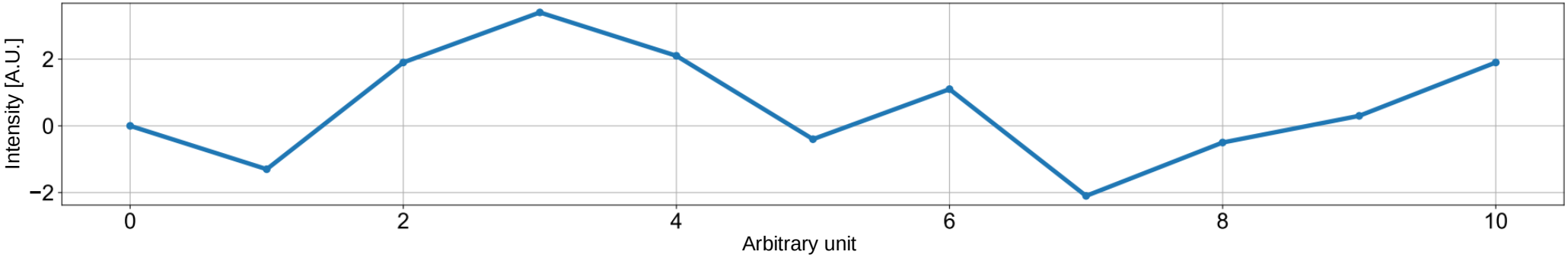


y	0	-1.3	1.9	3.4	2.1	-0.4	1.1	-2.1	-0.5	0.3	1.9

universität wien

DATA SMOOTHING

moving average smoothing

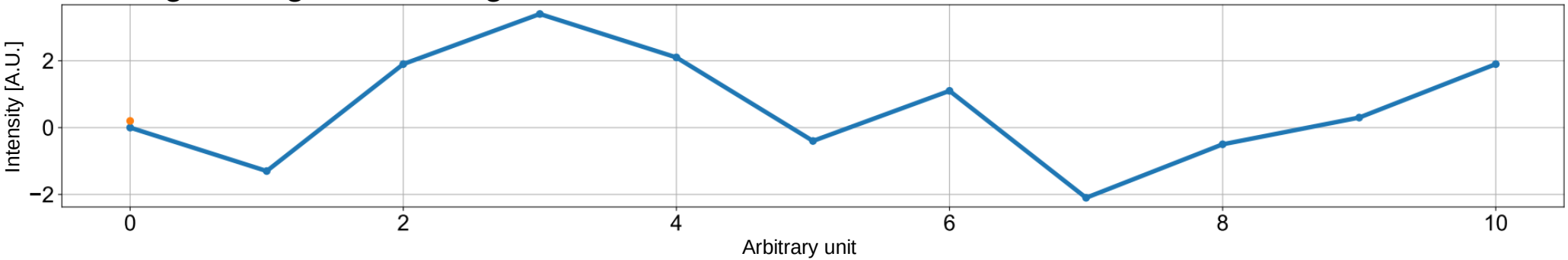


y	0	-1.3	1.9	3.4	2.1	-0.4	1.1	-2.1	-0.5	0.3	1.9

universität wien

DATA SMOOTHING

moving average smoothing



y	0	-1.3	1.9	3.4	2.1	-0.4	1.1	-2.1	-0.5	0.3	1.9

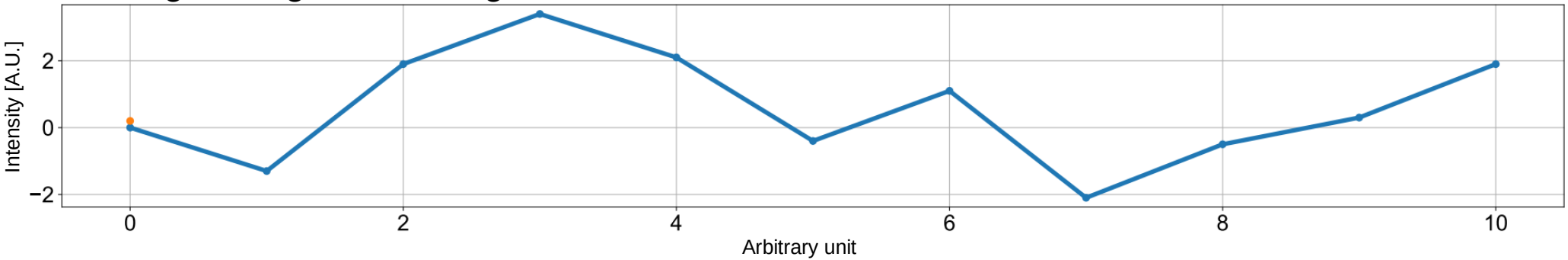
Ø

y’	0.2										

universität wien

DATA SMOOTHING

moving average smoothing



y	0	-1.3	1.9	3.4	2.1	-0.4	1.1	-2.1	-0.5	0.3	1.9

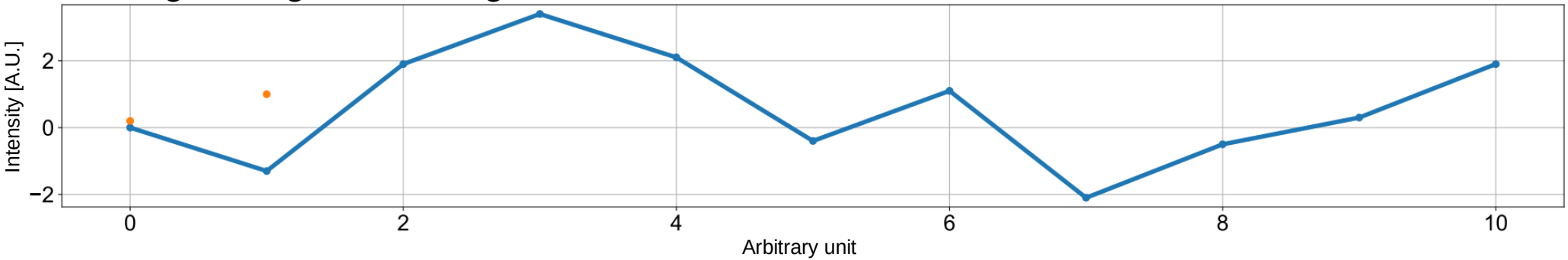
Ø

y’	0.2										

universität wien

DATA SMOOTHING

moving average smoothing



y	0	-1.3	1.9	3.4	2.1	-0.4	1.1	-2.1	-0.5	0.3	1.9

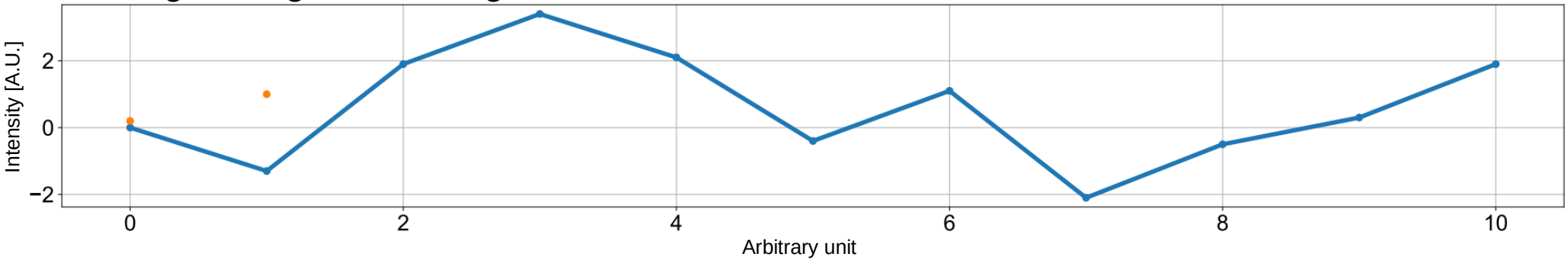
Ø

y’	0.2	1.0									

universität wien

DATA SMOOTHING

moving average smoothing



y	0	-1.3	1.9	3.4	2.1	-0.4	1.1	-2.1	-0.5	0.3	1.9

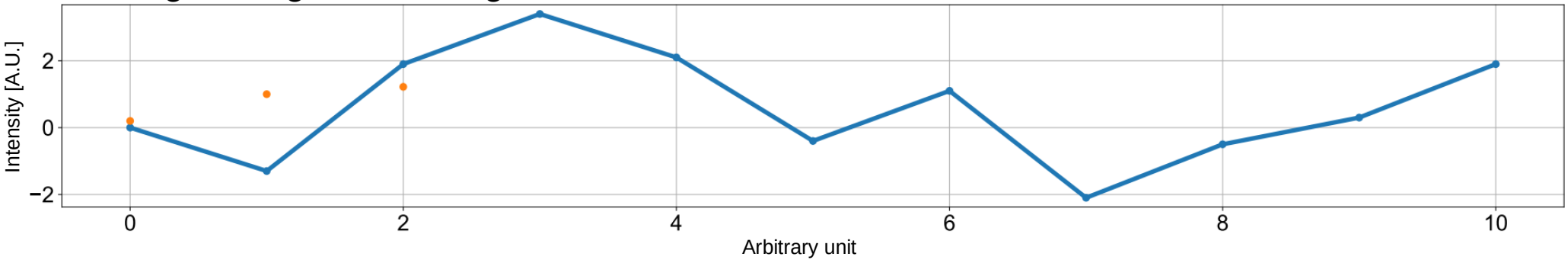
Ø

y’	0.2	1.0									

universität wien

DATA SMOOTHING

moving average smoothing



y	0	-1.3	1.9	3.4	2.1	-0.4	1.1	-2.1	-0.5	0.3	1.9

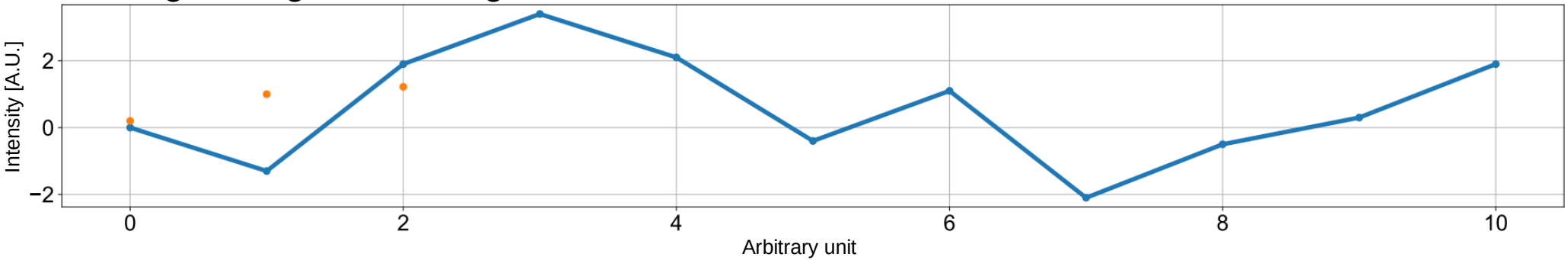
Ø

y’	0.2	1.0	1.22								

universität wien

DATA SMOOTHING

moving average smoothing



y	0	-1.3	1.9	3.4	2.1	-0.4	1.1	-2.1	-0.5	0.3	1.9

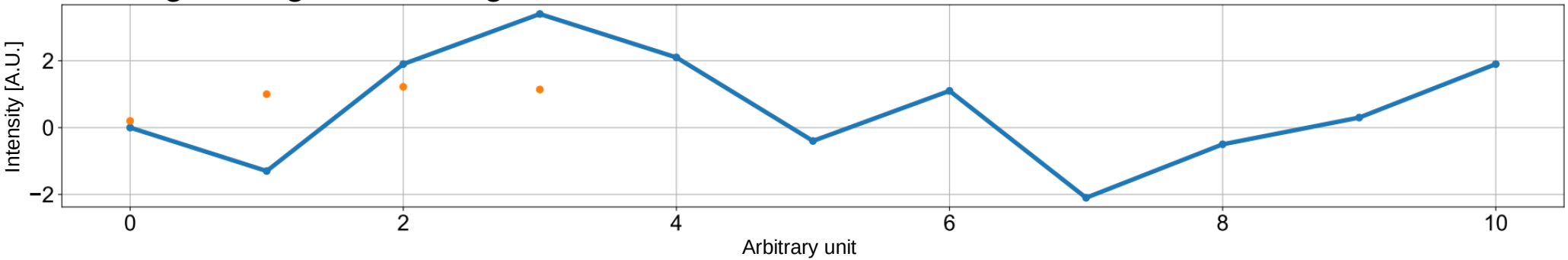
Ø

y’	0.2	1.0	1.22								

universität wien

DATA SMOOTHING

moving average smoothing



y	0	-1.3	1.9	3.4	2.1	-0.4	1.1	-2.1	-0.5	0.3	1.9

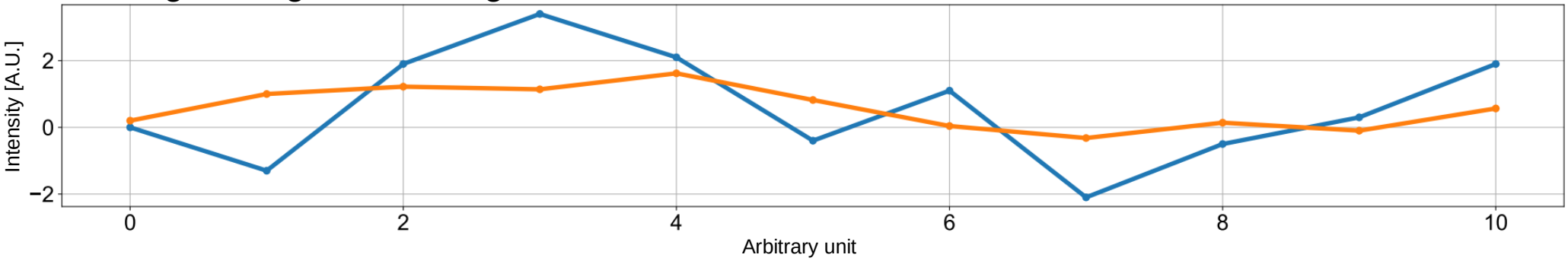
Ø

y’	0.2	1.0	1.22	1.14							

universität wien

DATA SMOOTHING

moving average smoothing



y	0	-1.3	1.9	3.4	2.1	-0.4	1.1	-2.1	-0.5	0.3	1.9

Ø

y’	0.2	1.0	1.22	1.14	1.62	0.82	0.04	-0.31	0.13	-0.1	0.56

universität wien

DATA SMOOTHING

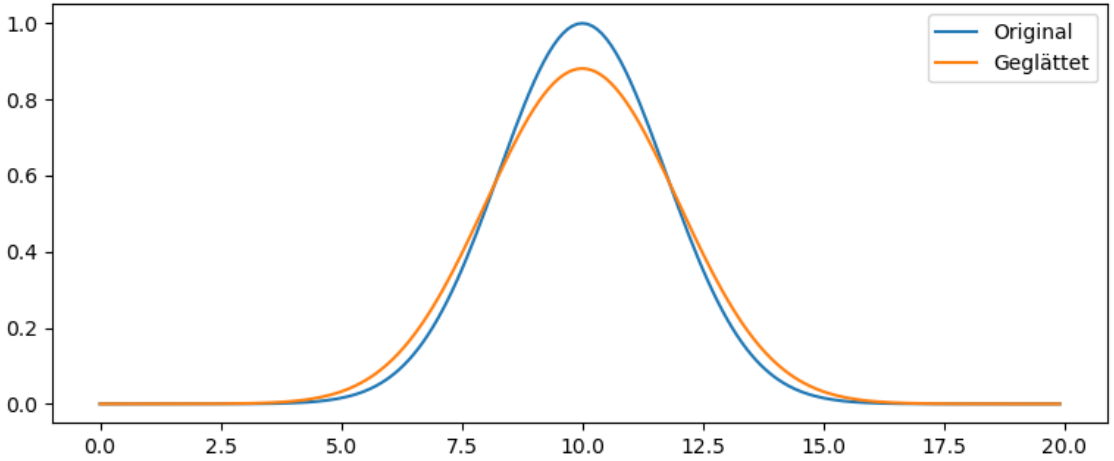
```
def moving_average_smoothing(x, y, window_width):
    y_new, st, en, n, hw = [], 0, 0, len(x), window_width / 2
    for i in range(n):
        while st < n - 1 and x[i] - x[st] > hw: st += 1
        while en < n - 1 and x[en] - x[i] <= hw: en += 1
        y_new.append(sum(y[st : en]) / (en - st + 1))
    return y_new
```

MOVING AVERAGE SMOOTHING

- Simple
- Good when data is very noisy
- Window size must be determined individually

MOVING AVERAGE SMOOTHING

- Simple
- Good when data is very noisy
- Window size must be determined individually
- Not suitable for little noisy data such as chromatograms or MS-spectra
- Peaks are suppressed

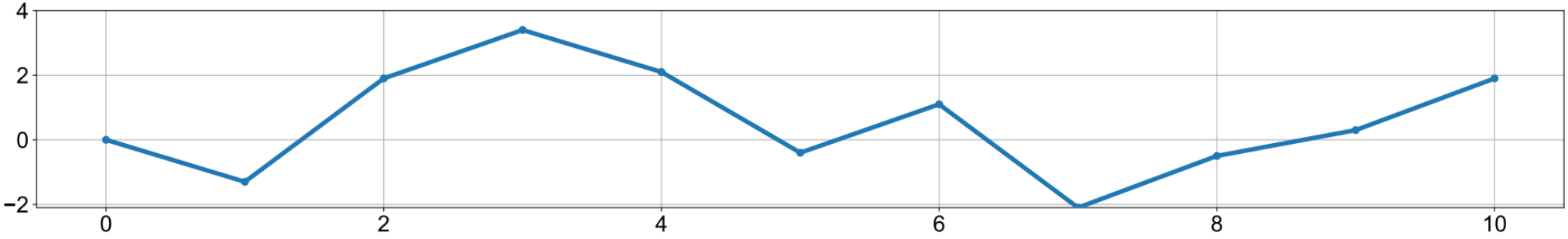


universität wien

SAVITZKY-GOLAY FILTER

- Smoothing filter from signal processing
- Tries to retain peak tip shape
- Suitable for little noisy data
- Idea: polynomial regression of data points in the local windows

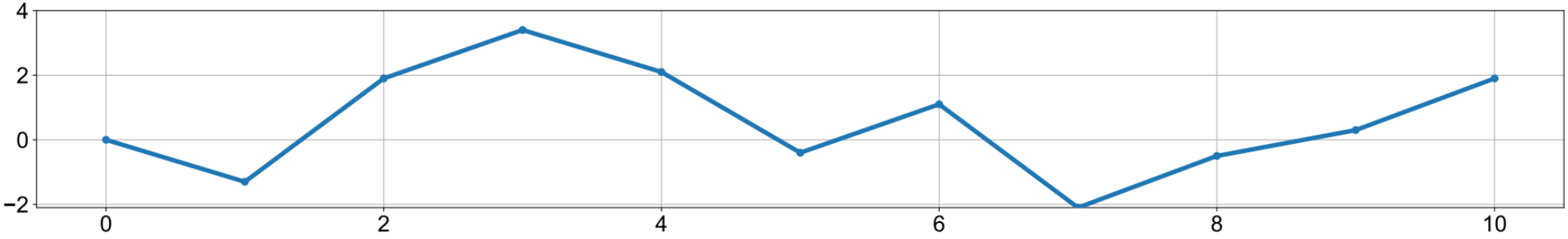
SAVITZKY-GOLAY FILTER



y	0	-1.3	1.9	3.4	2.1	-0.4	1.1	-2.1	-0.5	0.3	1.9

universität wien

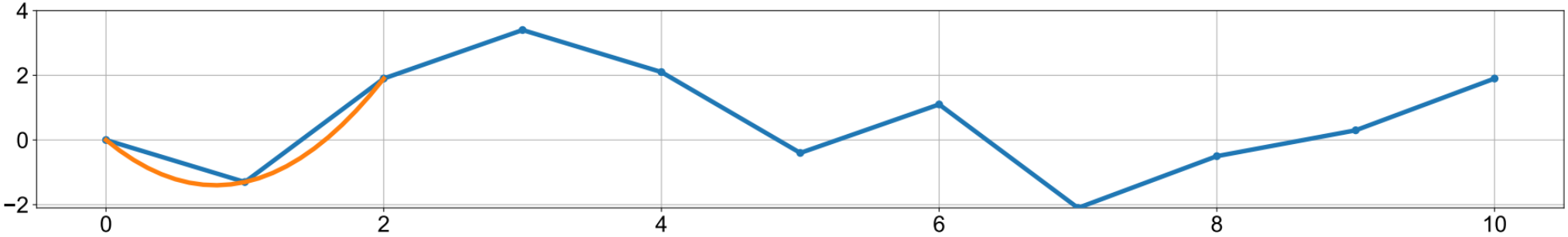
SAVITZKY-GOLAY FILTER



y	0	-1.3	1.9	3.4	2.1	-0.4	1.1	-2.1	-0.5	0.3	1.9

universität wien

SAVITZKY-GOLAY FILTER



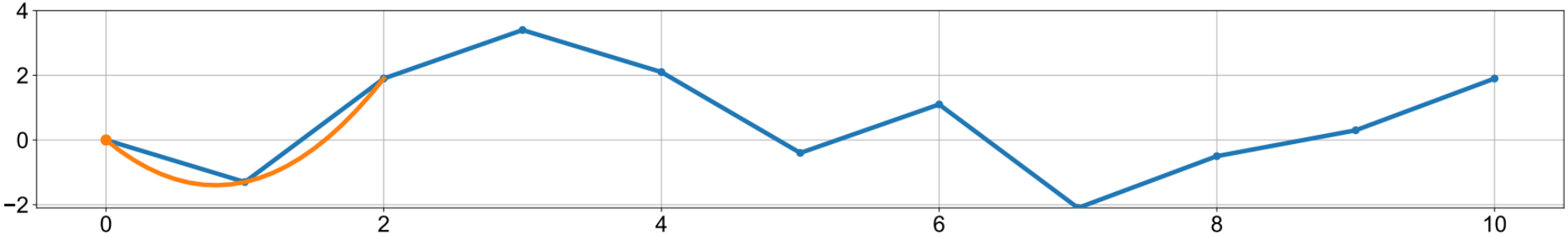
y	0	-1.3	1.9	3.4	2.1	-0.4	1.1	-2.1	-0.5	0.3	1.9

$$f(x) = 2.25\,x^2 - 3.55\,x$$

y’											

universität wien

SAVITZKY-GOLAY FILTER



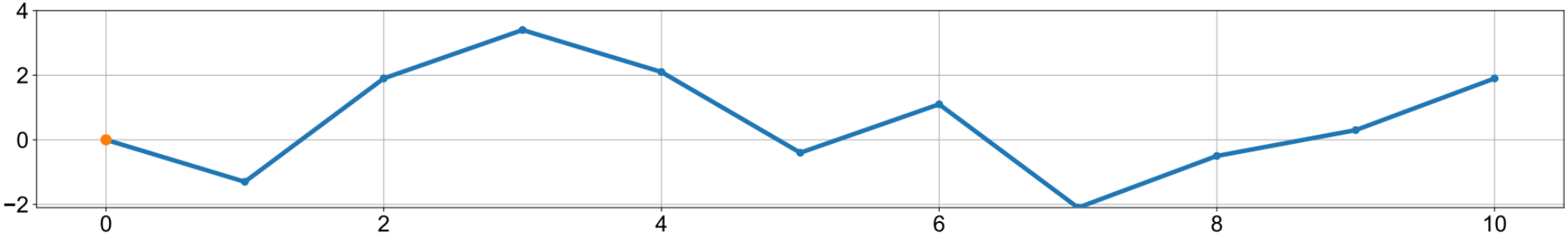
y	0	-1.3	1.9	3.4	2.1	-0.4	1.1	-2.1	-0.5	0.3	1.9

$$f(x)=2.25x^2-3.55x \qquad x=0 \Rightarrow f(x)=0$$

y’	0										

universität wien

SAVITZKY-GOLAY FILTER

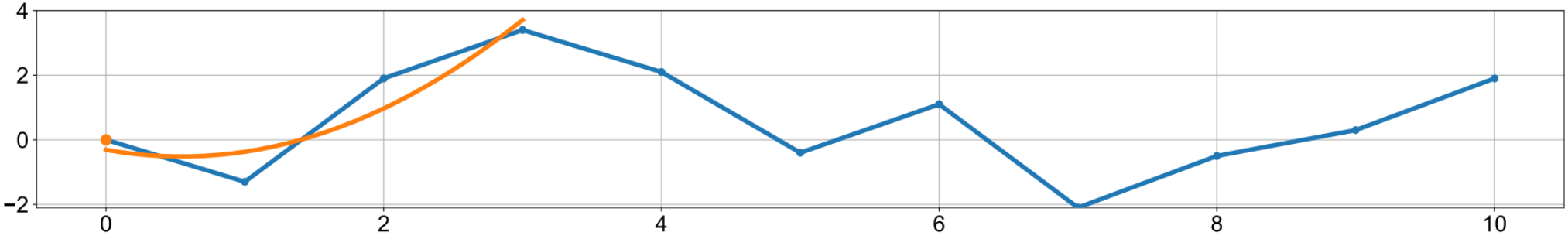


y	0	-1.3	1.9	3.4	2.1	-0.4	1.1	-2.1	-0.5	0.3	1.9

y’	0										

universität wien

SAVITZKY-GOLAY FILTER



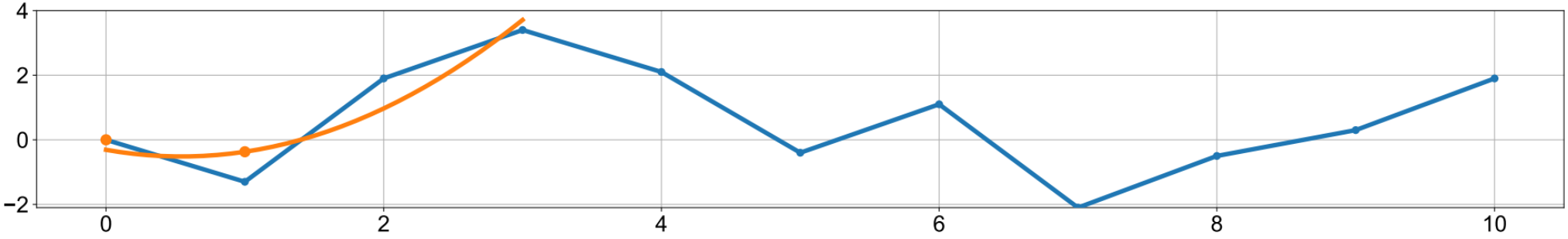
y	0	-1.3	1.9	3.4	2.1	-0.4	1.1	-2.1	-0.5	0.3	1.9

$$f(x) = 0.7\,x^2 - 0.76\,x - 0.31$$

y’	0										

universität wien

SAVITZKY-GOLAY FILTER



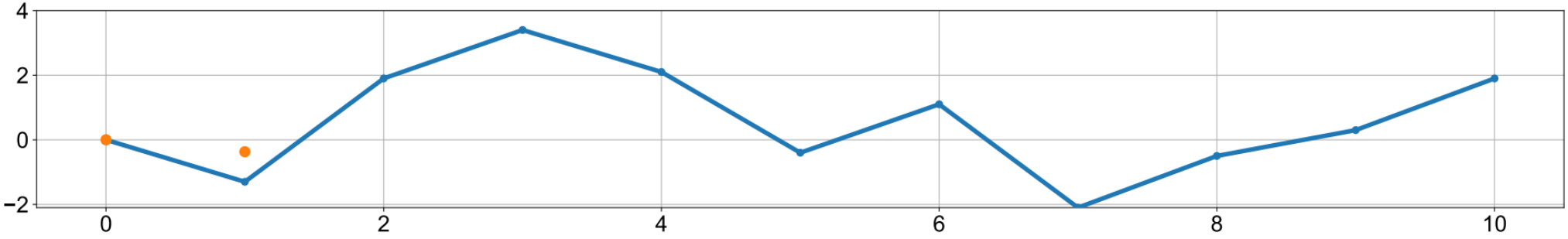
y	0	-1.3	1.9	3.4	2.1	-0.4	1.1	-2.1	-0.5	0.3	1.9

$$f(x)=0.7\,x^2-0.76\,x-0.31 \qquad x=1 \;\Rightarrow\; f(x)=-0.37$$

y’	0	-0.37									

universität wien

SAVITZKY-GOLAY FILTER

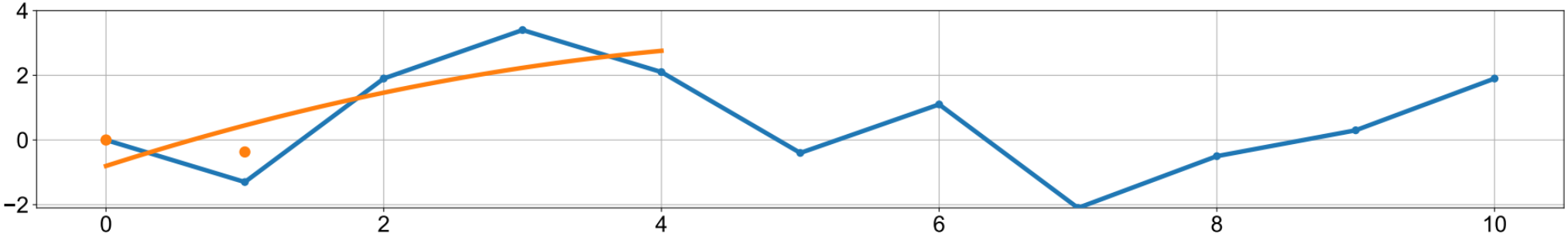


y	0	-1.3	1.9	3.4	2.1	-0.4	1.1	-2.1	-0.5	0.3	1.9

y’	0	-0.37									

universität wien

SAVITZKY-GOLAY FILTER



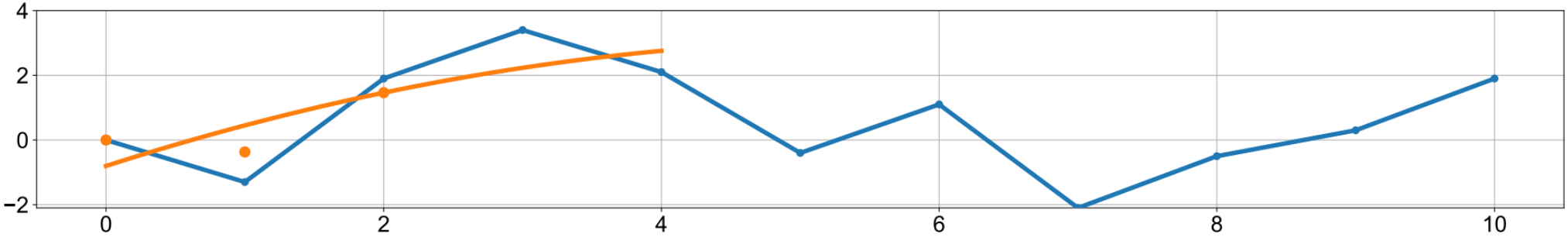
y	0	-1.3	1.9	3.4	2.1	-0.4	1.1	-2.1	-0.5	0.3	1.9

$$f(x) = -0.12\,x^2 + 1.37\,x - 0.80$$

y’	0	-0.37									

universität wien

SAVITZKY-GOLAY FILTER



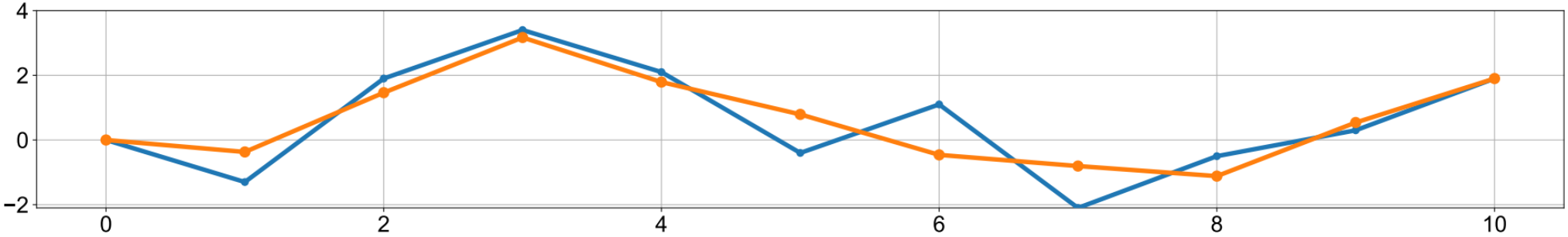
y	0	-1.3	1.9	3.4	2.1	-0.4	1.1	-2.1	-0.5	0.3	1.9

$$f(x) = -0.12\,x^2 + 1.37\,x - 0.80 \qquad x = 2 \Rightarrow f(x) = 1.46$$

y’	0	-0.37	1.46								

universität wien

SAVITZKY-GOLAY FILTER



y	0	-1.3	1.9	3.4	2.1	-0.4	1.1	-2.1	-0.5	0.3	1.9

y’	0	-0.37	1.46	3.16	1.79	0.79	-0.45	-0.8	-1.11	0.53	1.89

universität wien

DATA SMOOTHING WITH THE SAVITZKY-GOLAY FILTER

```
def sav_gol_filter(x, y, window_width):
    y_new, st, en, n, hw = [], 0, 0, len(x), window_width / 2
    for i in range(n):
        while st < n - 1 and x[i] - x[st] > hw: st += 1
        while en < n - 1 and x[en] - x[i] <= hw: en += 1
        param = np.polyfit(x[st : en], y[st : en], 2)
        y_new.append(param[0] * x[i]**2 + param[1] * x[i] + param[2])
    return y_new
```

COMPARISON OF SERIES

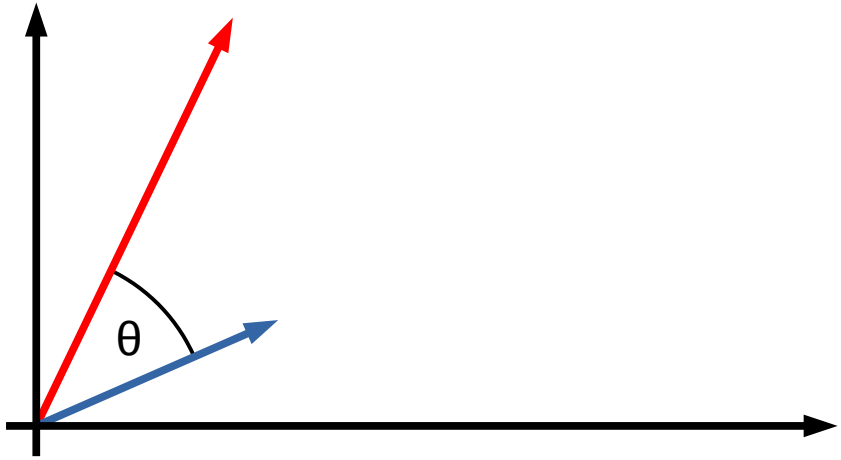
- Remainder: the hierarchical clustering is a distance-based approach
- To cluster our different series, we need to compute pairwise distances
- Requirement: the series need to have the same number of dimensions
  - A measured point at 5 min from the first time series should NOT be compared with a measured point at 7 min from the second time series

COMPARISON OF SERIES

- Remainder: the hierarchical clustering is a distance-based approach
- To cluster our different series, we need to compute pairwise distances
- Requirement: the series need to have the same number of dimensions
  - A measured point at 5 min from the first time series should NOT be compared with a measured point at 7 min from the second time series
- Idea: treat each dimension independently and use Manhattan oder Euclidean distance
- Other solution: Cosine similarity oder Pearson correlation

COSINE SIMILARITY

- Measure for similarity of two vectors from math
- An angle between both *n*-dimensional vectors is computed
- Length of the vectors is irrelevant



universität wien

COSINE SIMILARITY

- Measure can have these values:
  - -1 (opposing vector directions)
  - 0 (orthogonal, no similarity)
  - 1 (parallel vectors, full similarity)
  - Everything in between -1 and 1

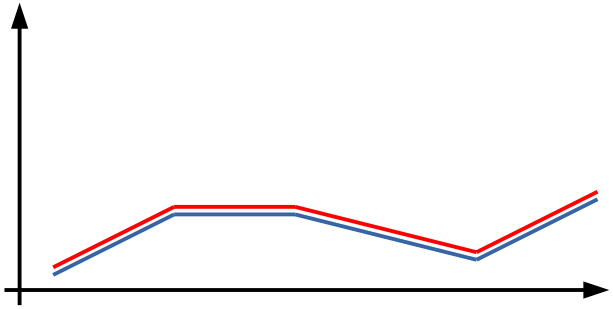
COSINE SIMILARITY

- Measure can have these values:
  - -1 (opposing vector directions)
  - 0 (orthogonal, no similarity)
  - 1 (parallel vectors, full similarity)
  - Everything in between -1 and 1
  - When the inpute values are all pure positive(or all negative), the measure reaches values between 0 and 1

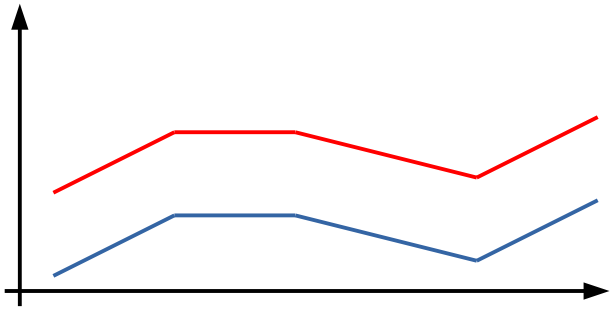
$$\cos(\theta) = \frac{a \cdot b}{\|a\| \, \|b\|} = \frac{\sum_{i=1}^{n} a_i \cdot b_i}{\sqrt{\sum_{i=1}^{n} a_i^2} \sqrt{\sum_{i=1}^{n} b_i^2}}$$

universität wien

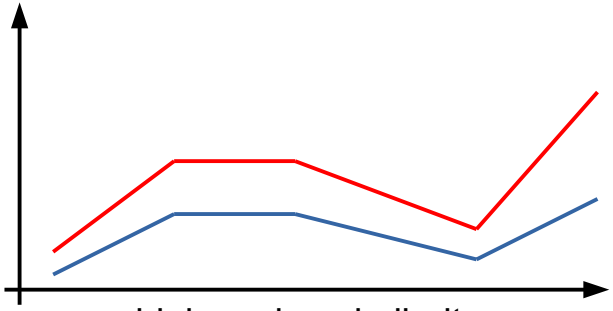
COSINE SIMILARITY, EXAMPLES



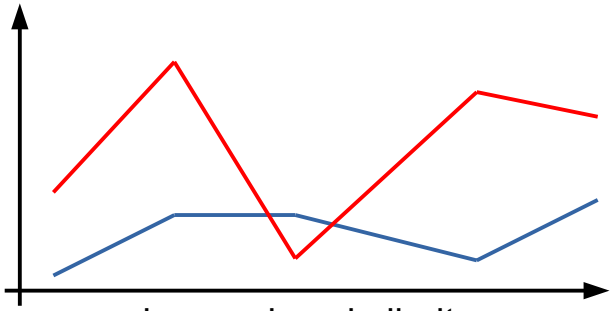
high cosine similarity



low cosine similarity



high cosine similarity

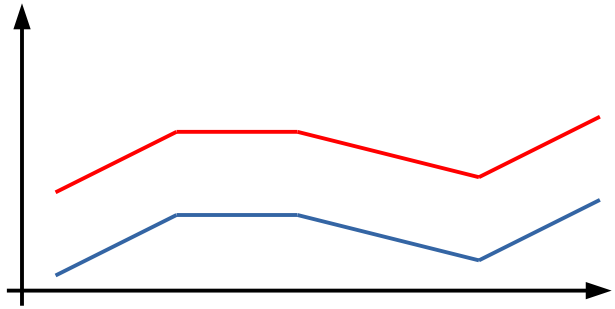


low cosine similarity

universität wien

PEARSON CORRELATION

- Two series, which are identical but shifted by a constant value in the y axis have a low cosine similarity

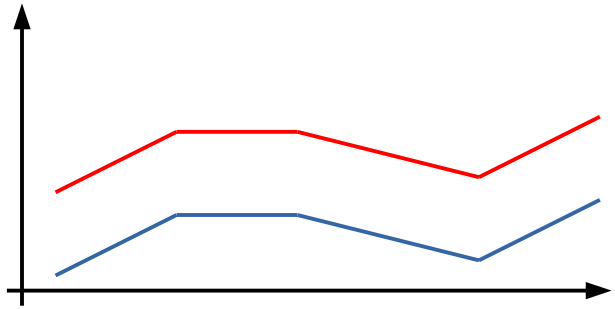


low cosine similarity

universität wien

PEARSON CORRELATION

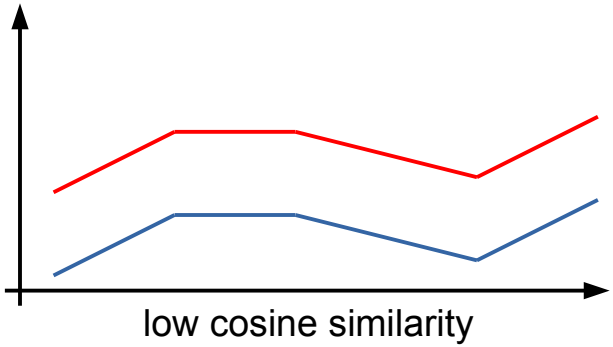
- Two series, which are identical but shifted by a constant value in the y axis have a low cosine similarity
- The Pearson correlation treats this case differently
- correlationen are different to similarities



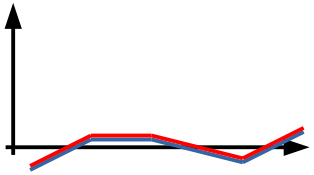
low cosine similarity

PEARSON CORRELATION

- Two series, which are identical but shifted by a constant value in the y axis have a low cosine similarity



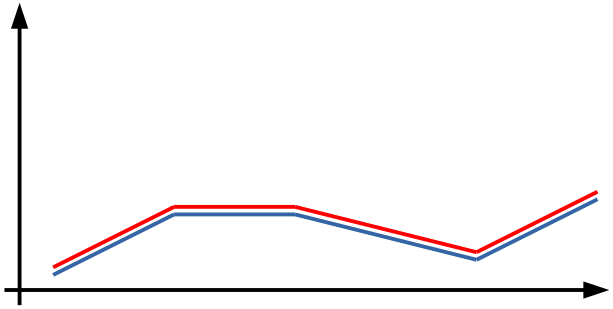
- The Pearson correlation treats this case differently
- correlationen are different to similarities
- Both series are shifted at the beginning to have mean value = 0



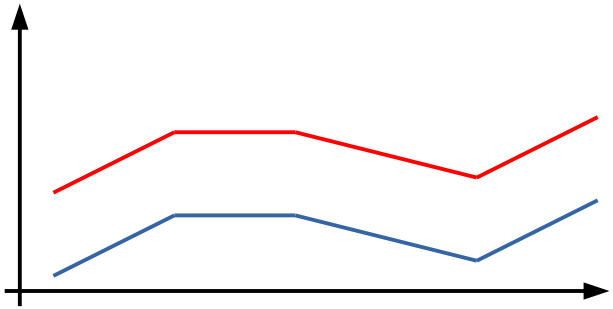
- Afterwards regular cosine similarity is performed

universität wien

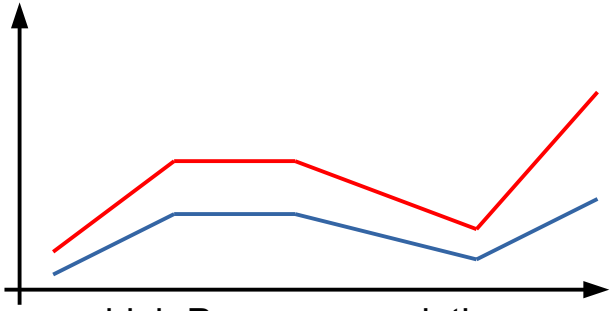
PEARSON CORRELATION, EXAMPLES



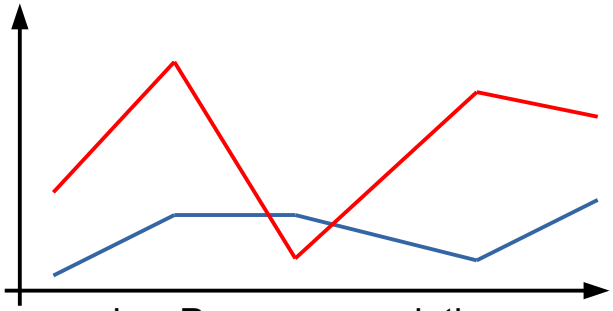
high Pearson correlation



high Pearson correlation



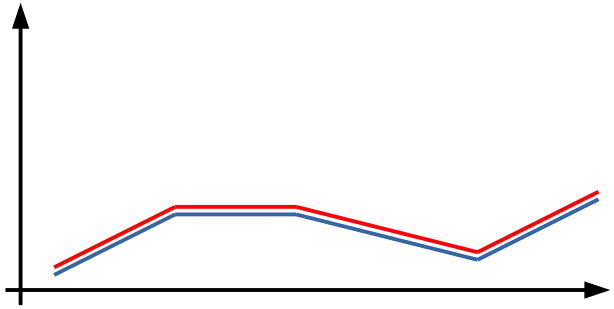
high Pearson correlation



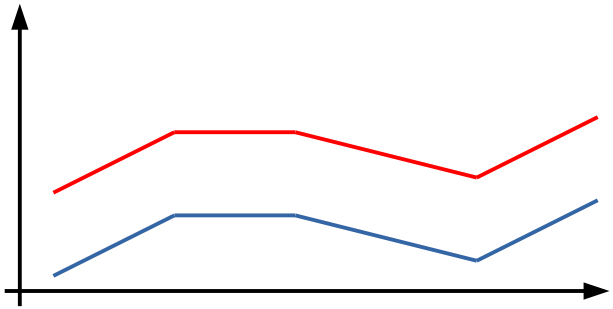
low Pearson correlation

universität wien

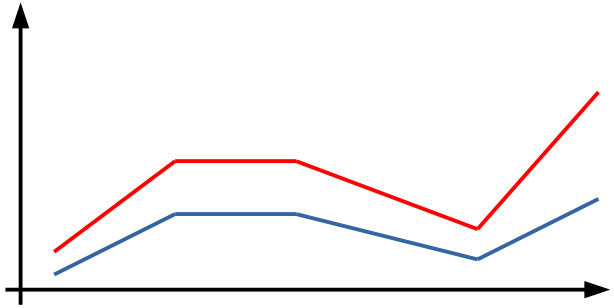
PEARSON CORRELATION, EXAMPLES



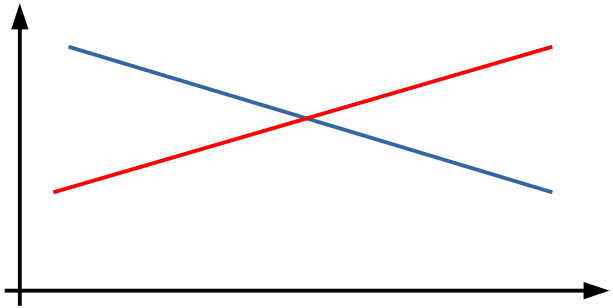
high Pearson correlation



high Pearson correlation



high Pearson correlation



negative Pearson correlation

Although we have only positive values, the Pearson correlationen can reach negative values!

universität wien

COMPREHENSIVE ANALYSIS OF ALL TIME SERIES

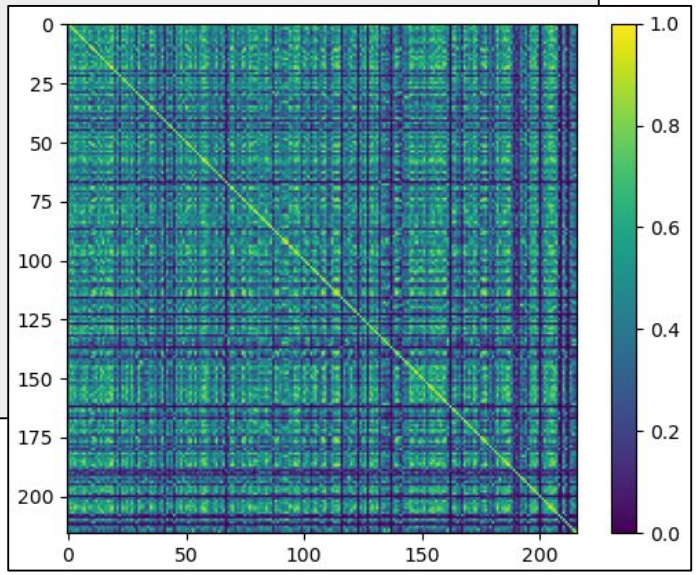
- Let‘s assume that our table has *n* time series (each series is in a row)
- To compare these series, we need to smooth them upfront
- Afterwards, we need to compute a pairwise distance matrix and visualize it

COMPREHENSIVE ANALYSIS OF ALL TIME SERIES

```
from scipy.spatial.distance import pdist, squareform

covid = pd.read_excel("covid-data-clean.xlsx", index_col = "location")
for i, row in covid.iterrows():
    covid.loc[i] = moving_average_smoothing(range(len(row)), row, 7)

D = 1 - squareform(pdist(covid, "cosine"))   # create a pairwise dist. matrix

cb = plt.imshow(D)
plt.colorbar(cb)
plt.show()
```

COMPREHENSIVE ANALYSIS OF ALL TIME SERIES

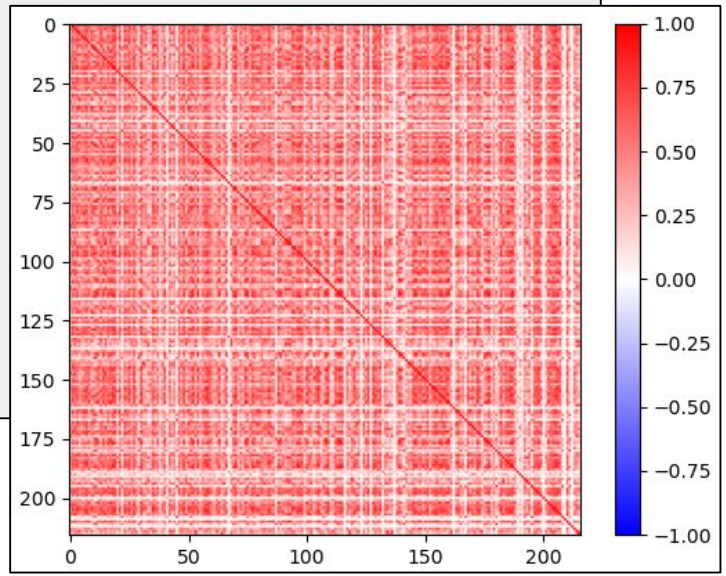
```
from scipy.spatial.distance import pdist, squareform

covid = pd.read_excel("covid-data-clean.xlsx", index_col = "location")
for i, row in covid.iterrows():
    covid.loc[i] = moving_average_smoothing(range(len(row)), row, 7)

D = 1 - squareform(pdist(covid, "cosine"))   # create a pairwise dist. matrix

cb = plt.imshow(D)
plt.colorbar(cb)
plt.show()
```



universität wien

COMPREHENSIVE ANALYSIS OF ALL TIME SERIES

```
from scipy.spatial.distance import pdist, squareform

covid = pd.read_excel("covid-data-clean.xlsx", index_col = "location")
for i, row in covid.iterrows():
    covid.loc[i] = moving_average_smoothing(range(len(row)), row, 7)

D = 1 - squareform(pdist(covid, "cosine"))   # create a pairwise dist. matrix

cb = plt.imshow(D, cmap = "bwr", vmin = -1, vmax = 1)
plt.colorbar(cb)
plt.show()
```



universität wien

COMPREHENSIVE ANALYSIS OF ALL TIME SERIES

- The computed distance matrix looks very weird and we cannot recognize here anything
- The rows and columns need to be sorted that similar rows are being put together

COMPREHENSIVE ANALYSIS OF ALL TIME SERIES

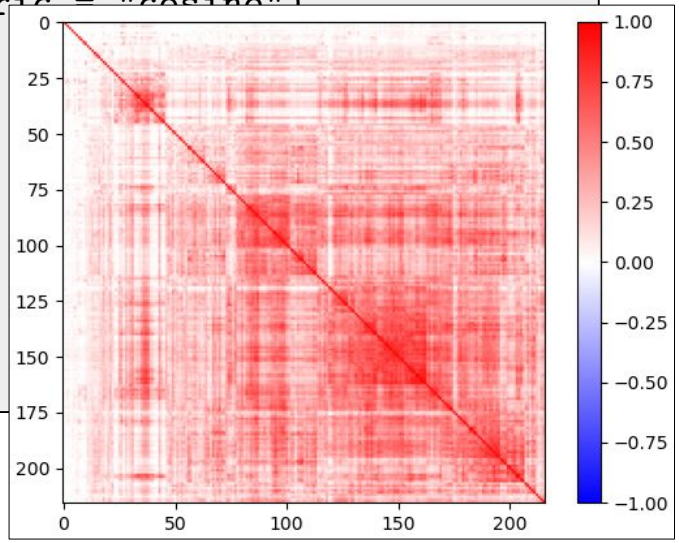
- The computed distance matrix looks very weird and we cannot recognize here anything
- The rows and columns need to be sorted that similar rows are being put together
- The sorting can be again performed with hierarichical clustering, as discussed in the previous lecture

COMPREHENSIVE ANALYSIS OF ALL TIME SERIES

```
covid = pd.read_excel("covid-data-clean.xlsx", index_col = "location")
for i, row in covid.iterrows():
    covid.loc[i] = moving_average_smoothing(range(len(row)), row, 7)
# create a pairwise dist. matrix for visualization
D = 1 - squareform(pdist(covid, "cosine"))

# create a clustering and sort the observations accordingly
lkg = hierarchy.linkage(covid, method = 'average', metric = "cosine")
labels_sorting = hierarchy.leaves_list(lkg)

D = D[labels_sorting]      # rowwise sorting
D = D[:, labels_sorting]   # columnwise Sorting
cb = plt.imshow(D, cmap = "bwr", vmin = -1, vmax = 1)
plt.colorbar(cb)
plt.show()
```

COMPREHENSIVE ANALYSIS OF ALL TIME SERIES

```
covid = pd.read_excel("covid-data-clean.xlsx", index_col = "location")
for i, row in covid.iterrows():
    covid.loc[i] = moving_average_smoothing(range(len(row)), row, 7)
# create a pairwise dist. matrix for visualization
D = 1 - squareform(pdist(covid, "cosine"))

# create a clustering and sort the observations accordingly
lkg = hierarchy.linkage(covid, method = 'average', metric = "cosine")
labels_sorting = hierarchy.leaves_list(lkg)

D = D[labels_sorting]      # rowwise sorting
D = D[:, labels_sorting]   # columnwise Sorting
cb = plt.imshow(D, cmap = "bwr", vmin = -1, vmax = 1)
plt.colorbar(cb)
plt.show()
```



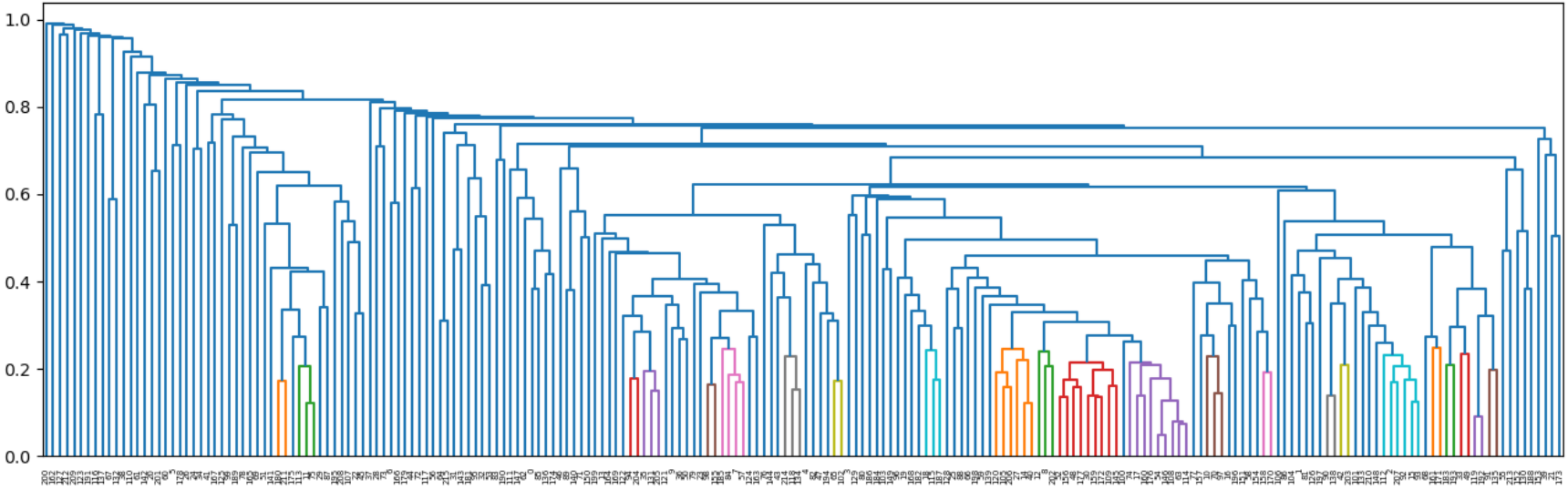
universität wien

GROUPING OF TIME SERIES

- The hierarchical clustering can be used, to group the time series
- We want to have groups of time series that each series has at least a similarity of x % to all others within a group
- Here, we can apply the knowledge we got from the previous lecture, let‘s have a look again

GROUPING OF TIME SERIES

```
lkg = hierarchy.linkage(covid, method = 'average', metric = "cosine")
# pairwise similarity of at least 75% is 1 - 0.75 = 0.25 distance
dn = hierarchy.dendrogram(lkg, color_threshold = 0.25)
plt.show()
```



GROUPING OF TIME SERIES

```
# create a cluster list
clusters = hierarchy.fcluster(lkg, t = 0.25, criterion = 'distance')
print(clusters)
# [50 123 127  88  86  10  36  70 103 ... 28 161 129  18   2 142  79  41]

# create a dictionary: group index → list of observation row indexes
groups = {c: [] for c in set(clusters)}
for i, c in enumerate(clusters): groups[c].append(i)

# print groups in the console
for k, v in groups.items():
    print(k, [covid.index[i] for i in v])
```

GROUPING OF TIME SERIES

```
...
19 ['Austria', 'Germany', 'Liechtenstein']
70 ['Argentina', 'Dominican Republic', 'Guyana', 'Suriname']]
102 ['Bahrain', 'Brazil', 'Chile', 'Kuwait', 'Maldives', 'Uruguay']
103 ['Armenia', 'Azerbaijan', 'Ukraine']
104 ['Bulgaria', 'Croatia', 'Czechia', 'Lebanon', 'North Macedonia', 'Poland',
'Romania', 'Serbia', 'Slovenia']
105 ['Belarus', 'Denmark', 'Estonia', 'Georgia', 'Latvia', 'Lithuania', 'Norway',
'Russia', 'Slovakia']]
127 ['Algeria', 'Bangladesh', 'Iran', 'Iraq', 'Libya', 'Uzbekistan']
...
```

THE BIG PICTURE OF THE TIME SERIES ANALYSIS

- We have different components, that we can visualize:
  - Heatmap
  - Hierarchical clustering
  - Grouping
- It would be awesome, when everything could be united in one figure
- The function `clustermap` from the `seaborn` library can be used for that

THE BIG PICTURE OF THE TIME SERIES ANALYSIS

```
import seaborn as sns      # import Seaborn library

covid = pd.read_excel("covid-data-clean.xlsx", index_col = "location")
for i, row in covid.iterrows():
    covid.loc[i] = moving_average_smoothing(range(len(row)), row, 7)

D = 1 - squareform(pdist(covid, "cosine"))   # create pairwise dist. matrix

cm = pd.DataFrame(D, index = covid.index, columns = covid.index)
sns.clustermap(cm, method = "average", metric = "cosine", cmap = "bwr",
vmin = -1, vmax = 1)
plt.show()
```

THE BIG PICTURE OF THE TIME SERIES ANALYSIS

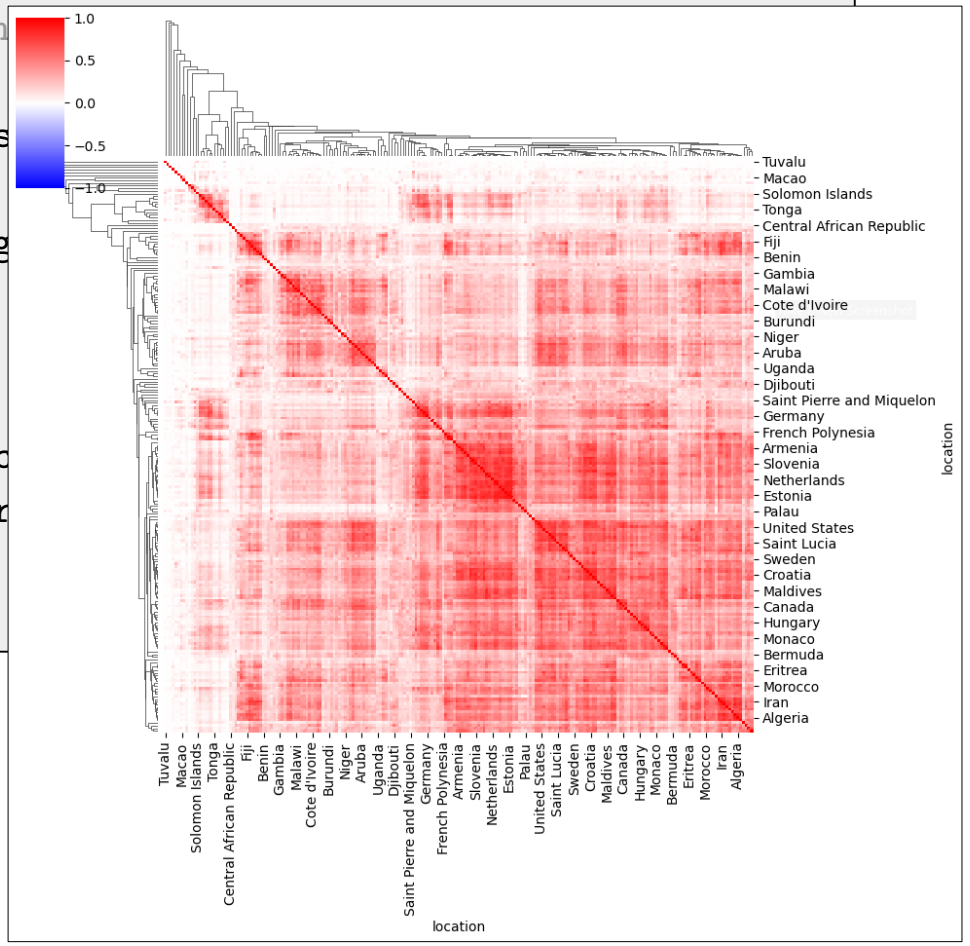
```
import seaborn as sns      # import Seaborn library

covid = pd.read_excel("covid-data-clean.xlsx", index_col = "location")
for i, row in covid.iterrows():
    covid.loc[i] = moving_average_smoothing(range(len(row)), row, 7)

D = 1 - squareform(pdist(covid, "cosine"))   # create pairwise dist. matrix

cm = pd.DataFrame(D, index = covid.index, columns = covid.index)
sns.clustermap(cm, method = "average", metric = "cosine", cmap = "bwr",
vmin = -1, vmax = 1)
plt.show()
```

Different tasks which use the same metric but don‘t need to

universität wien

THE BIG PICTURE OF THE TIME SERIES ANALYSIS

```
import seaborn as sns      # import Seaborn

covid = pd.read_excel("covid-data-clean.xls
for i, row in covid.iterrows():
    covid.loc[i] = moving_average_smoothing

D = 1 - squareform(pdist(covid, "cosine"))

cm = pd.DataFrame(D, index = covid.index, c
sns.clustermap(cm, method = "average", metr
vmin = -1, vmax = 1)
plt.show()
```



universität wien

THE BIG PICTURE OF THE TIME SERIES ANALYSIS

- The grouping has to be added yet
- But it is dependent on the sorting of the clustering

THE BIG PICTURE OF THE TIME SERIES ANALYSIS

- The grouping has to be added yet
- But it is dependent on the sorting of the clustering
- We applied two different clusterings from different libraries:
  - with `hierarchy.linkage`
  - with `sns.clustermap`
- Since we determine the grouping with the result of `hierarchy.linkage`, we insert it again in `sns.clustermap`

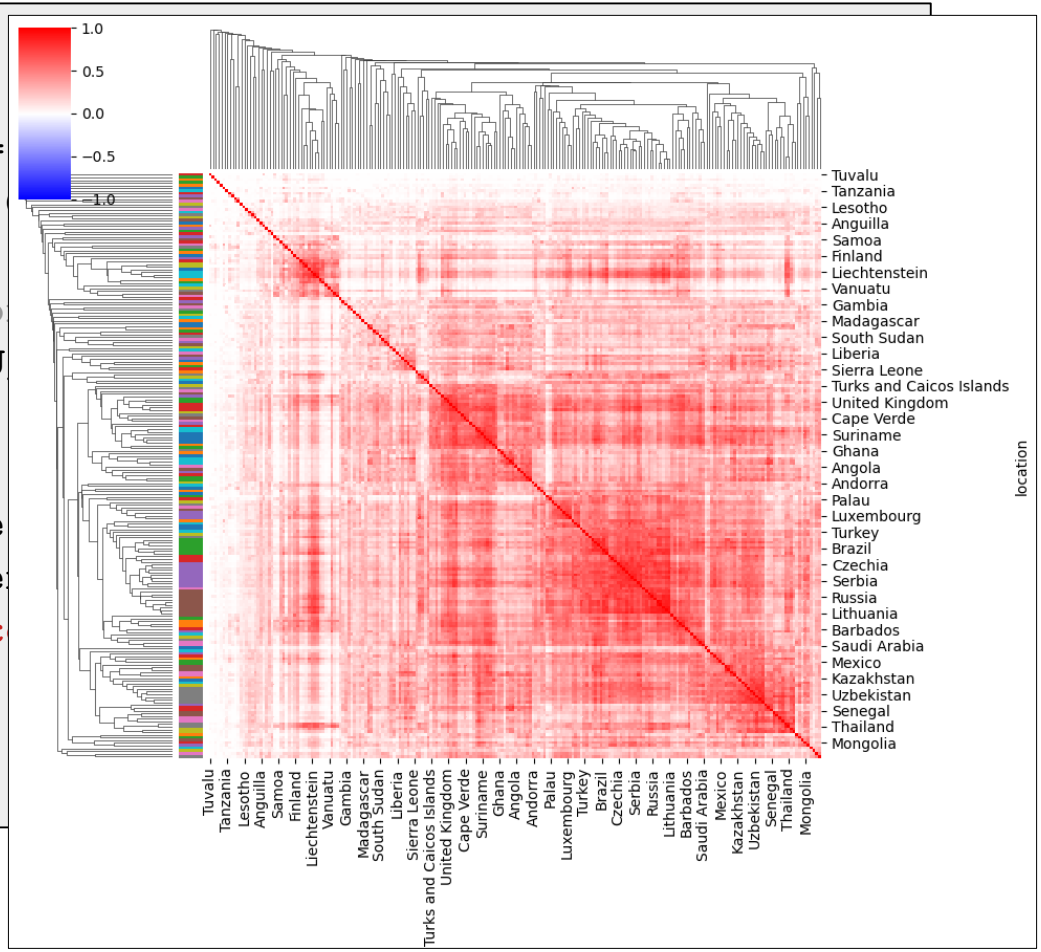
THE BIG PICTURE OF THE TIME SERIES ANALYSIS

```
import matplotlib.colors as mcolors

lkg = hierarchy.linkage(covid, method = 'average', metric = "cosine")
clusters = hierarchy.fcluster(lkg, t = 0.25, criterion = 'distance')

# create a list of colors independent of the time series groups
colors = [v for k, v in mcolors.TABLEAU_COLORS.items()]
group_colors = [colors[c % len(colors)] for c in clusters]

D = 1 - squareform(pdist(covid, "cosine"))
cm = pd.DataFrame(D, index = covid.index, columns = covid.index)
sns.clustermap(cm, row_linkage = lkg, col_linkage = lkg, cmap = "bwr", vmin = -1,
vmax = 1, row_colors = group_colors)

plt.show()
```

universität wien

THE BIG PICTURE OF THE TIME SERIES ANALYSIS

```
import matplotlib.colors as mcolors

lkg = hierarchy.linkage(covid, method =
clusters = hierarchy.fcluster(lkg, t =

# create a list of colors independent o
colors = [v for k, v in mcolors.TABLEAU
group_colors = [colors[c % len(colors)]

D = 1 - squareform(pdist(covid, "cosine
cm = pd.DataFrame(D, index = covid.inde
sns.clustermap(cm, row_linkage = lkg, c
vmax = 1, row_colors = group_colors)

plt.show()
```



universität wien